

3-adischer Steuerbaum mit Blockchain-Hash und Quellcode

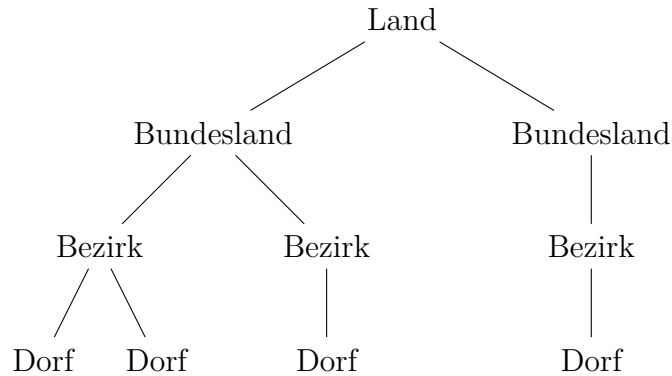
Alexander Kern

January 9, 2026

1 Hierarchie des Baums

Wir modellieren eine föderale Struktur als **3-adischen Baum**:

- Ebene 0: Dörfer (1–3 pro Bezirk)
- Ebene 1: Bezirke (1–3 pro Bundesland)
- Ebene 2: Bundesländer (1–3 pro Land)
- Ebene 3: Länder (1–3 insgesamt)



2 Blockchain-Hash für jeden Knoten

Jeder Knoten erhält einen Hash, der von seinem Namen, den Hashes seiner Kinder und dem Hash seines Elternknotens abhängt:

$$\text{hash}_{\text{node}} = H\left(\text{name}_{\text{node}} \parallel \text{parent_hash} \parallel \parallel_{c \in \text{children}} \text{hash}_c\right) \quad (1)$$

3 Steueraggregation

3.1 Dorfebene

Jedes Dorf erhebt eine lokale Steuer:

$$\text{tax}_{\text{dorf}} = \text{Einwohner} \times \text{Steuersatz} \quad (2)$$

3.2 Aggregation nach oben

Die Steuer auf jeder Ebene ist die Summe der Steuern aller Kinder:

$$\text{tax}_{\text{node}} = \text{tax}_{\text{local}} + \sum_{c \in \text{children}} \text{tax}_c \quad (3)$$

3.3 Verteilung auf EU-Länder

Am Ende kann die Steuer des Landes auf 1–3 EU-Länder verteilt werden:

$$\text{tax}_{\text{EU_Land}} = \frac{\text{tax}_{\text{Land}}}{n_{\text{EU-Länder}}}, \quad 1 \leq n_{\text{EU-Länder}} \leq 3 \quad (4)$$

4 Algorithmus: Baum erzeugen mit Hashes und Steuern

Algorithm 1 Erzeuge 3-adischen Steuerbaum mit Blockchain-Hash

- 1: **Input:** Ebenen: Land $\rightarrow$ Bundesland $\rightarrow$ Bezirk $\rightarrow$ Dorf
 - 2: **Function** GenerateNode(level, depth, parentHash)
 - 3: Erzeuge Name für Knoten
 - 4: Falls Dorf: Steuer lokal zufällig setzen
 - 5: Falls depth > 0 :
 - 6: Wähle 1–3 Kinder
 - 7: Rekursiv: GenerateNode(childLevel, depth-1, parentHash)
 - 8: Berechne node.hash = Hash(Name $\parallel$ parentHash $\parallel$ Kinderhashes)
 - 9: Berechne node.totalTax = tax_local + Summe Kinder
 - 10: **return** node
-

5 Implementierungen in C++, Java und Haskell

5.1 C++

```
#include <iostream>
#include <vector>
#include <string>
#include <openssl/sha.h>
struct Node {
    std::string name;
    double localTax = 0.0;
    double totalTax = 0.0;
    std::vector<Node> children;
};
double computeTax(Node& node){
    double sum = node.localTax;
    for (auto& c : node.children)
        sum += computeTax(c);
    node.totalTax = sum;
    return sum;
}
```

5.2 Java

```
class Node {
    String name;
    double localTax;
    double totalTax;
    List<Node> children = new ArrayList<>();
    double computeTax(){
        totalTax = localTax;
        for(Node c : children) totalTax += c.computeTax();
        return totalTax;
    }
}
```

5.3 Haskell

```
data Node = Node { name :: String, localTax :: Double, children :: [Node]
```

```

totalTax :: Node -> Double
totalTax n = localTax n + sum (map totalTax (children n))
distributeEU :: Double -> [Node] -> [(String, Double)]
distributeEU tax countries =
    let share = tax / fromIntegral (length countries)
    in map (\c -> (name c, share)) countries

```

6 Fazit

Wir haben ein föderales Steuer-System modelliert, das:

- 3-adisch aufgebaut ist
- Dörfer Steuern erheben, die nach oben aggregiert werden
- Blockchain-Hashes die Integrität sichern
- Am Ende die Steuern optional auf EU-Länder verteilen kann
- Implementierungen in C++, Java und Haskell vorliegen